

# 通行止め・通行困難スポット表示機能 設計書

バージョン: 0.1（ドラフト） 最終更新日: 2026年6月20日 ステータス: 設計検討中（実装は未着手）

## 1. 概要

### 1.1 目的

ハイキングマップ上に、工事による通行止めや、倒木・落石等による通行困難箇所を「スポット」として重ね合わせ表示する。対象地点は運用担当者が登録・判断し、テスト（確認）を経て一般ユーザーへ公開する運用を可能にする。

### 1.2 背景・前提

- 通行止め／通行困難は一時的・頻繁に変化し、解除（削除）も必要な情報である。既存の geojson（緊急ポイント・ルート/スポット）と異なり、即時の反映・解除が求められる。
- 地点データの\*\*作成は別アプリ（TrailMapper系・ファイル出力のみ）\*\*で行う。Firebase は使用しない。
- 登録・公開は**運用担当者**が行い、開発担当の作業（git commit / deploy 等）は介在させない。ただし運用担当者の操作が**1ボタン/1コマンドで完結**するなら許容する。

### 1.3 スcope

- 本書は表示機能と「テスト→公開」の配信方式（後述の **P2**）の設計を対象とする。
- 地点データを作成する別アプリ（TrailMapper系）側の改修は本書の対象外。
- 後述「9. 保留事項」は運用担当者との協議後に別途設計する。

### 1.4 用語

| 用語          | 意味                                                |
|-------------|---------------------------------------------------|
| closures    | 本機能で扱う通行止め・通行困難スポットの総称                            |
| draft       | テスト中（一般ユーザー非公開）の地点                                |
| published   | 一般公開済みの地点                                         |
| preview モード | 運用担当者が draft を含めて確認するための表示モード                     |
| 公開ストア       | 全ユーザーの端末が取得しに行く共有の保存先（本設計では同一オリジンの API + Blob/KV） |

## 2. 配信方式の決定（P2）

### 2.1 検討した選択肢

| 案                | 内容                                                 | 採否                                                |
|------------------|----------------------------------------------------|---------------------------------------------------|
| アプリシェル同梱         | 既存 geojson と同様に <code>SHELL_LOCAL_PATHS</code> に同梱 | × 変更ごとに再デプロイ+全ユーザーの更新プロンプトが必要で重い                  |
| P1: 1コマンドgit公開   | 検証後にスクリプトで commit→push→Vercel自動デプロイ                | △ 新インフラ不要だが、 <b>アプリ内ボタンでは実行不可</b> （ブラウザに git が無い） |
| P2: 同一オリジンの公開API | 自プロジェクトの Vercel Function 経由で公開ストアへ保存               | ◎ 採用                                              |
| P3: Firebase     | Firestore に保存=公開                                   | × Firebase 不使用のため対象外                              |

## 2.2 採用理由（P2）

- 「**preview 時のみ表示される公開ボタン**」で公開を完結したいという要件を満たせる唯一の方式。ブラウザ内のボタンは git push を実行できないため、押下先となる**同一オリジンの API が必須**となる。
- 外部サービス依存ではなく**自プロジェクト（Vercel）自身の API**であり、データ更新は **git / deploy を一切経由しない**。運用担当者の端末に git は不要。
- 同一オリジンのため **CORS 設定が不要**。Service Worker でのオフラインキャッシュも容易。

## 2.3 公開の本質的制約（記録）

静的ホスティングの PWA で全ユーザーに見せるには、全端末が取得しに行く**共有の置き場**が必須。置き場は「(a) 自配信元(Vercel)」か「(b) 外部ストア」の二択であり、両方を避けると全公開は成立しない。P2 は (a) に該当し、運用担当者の操作を 1ボタンに包むことでこの制約を満たす。

## 3. アーキテクチャ全体像

```

[別アプリ TrailMapper系]
  | geojson 出力 (status 付き・status の設定方法は「9. 保留事項」)
  ▼
[運用担当者の端末ブラウザ：本PWA を ?preview=closures で表示]
  | ① ファイル取り込み（端末ローカル / IndexedDB）
  | ② 地図上で検証
  | ③ 「公開」ボタン（preview 時のみ表示）
  ▼ POST /api/closures（公開トークン認証）
[Vercel Function + 公開ストア（Vercel Blob または KV）]
  ▲ GET /api/closures（全ユーザーがフレッシュ取得）
  |
[一般ユーザーの端末：本PWA]
  | status==='published' のみ地図に描画（オフライン時は最終取得をキャッシュ表示）

```

## 4. データ仕様

### 4.1 ファイル／配信

- 既存 `minoh-hiking-routes-spots.geojson` とは**別ファイル**として扱う。

- 配信元は公開ストアであり、本アプリは API (GET /api/closures) 経由で取得する。
- アプリシェル (SHELL\_CACHE) には同梱しない (再デプロイ・更新プロンプト無しで反映するため)。

## 4.2 GeoJSON スキーマ

FeatureCollection。各 Feature は Point (スポット)。

| プロパティ        | 型      | 必須 | 説明                                   |
|--------------|--------|----|--------------------------------------|
| type         | string | ✓  | 固定値 "closure"                        |
| id           | string | ✓  | 地点の一意識別子 (例 C-01)。全地点で一意             |
| name         | string | ✓  | 名称 (例「風呂谷 倒木」)                       |
| kind         | string | ✓  | "closed" (通行止め) / "difficult" (通行困難) |
| reason       | string | -  | 理由 (工事 / 倒木 / 落石 等)                  |
| status       | string | ✓  | "draft" (テスト中) / "published" (公開)    |
| note         | string | -  | 補足 (例「巻き道あり」)                        |
| relatedRoute | string | -  | 関連ルートID (将来のナビ連携用・任意)                |
| updatedAt    | string | -  | 更新日 (ISO 8601 / YYYY-MM-DD)          |

geometry: { "type": "Point", "coordinates": [経度, 緯度(, 標高)] }

## 4.3 例

```
{
  "type": "FeatureCollection",
  "updatedAt": "2026-06-20T10:00:00+09:00",
  "features": [
    {
      "type": "Feature",
      "properties": {
        "type": "closure",
        "id": "C-01",
        "name": "風呂谷 倒木",
        "kind": "difficult",
        "reason": "倒木",
        "status": "draft",
        "note": "巻き道あり",
        "relatedRoute": "R-12",
        "updatedAt": "2026-06-20"
      },
      "geometry": { "type": "Point", "coordinates": [135.476, 34.850] }
    }
  ]
}
```

## 4.4 公開可否と可視性

- 一般ユーザー：`status === "published"` の地点のみ描画する。
- preview モード：`draft` を含む全地点を描画する（`draft` は警告色等で区別）。
- **`draft` が生 JSON（API 応答）に含まれ、一般ユーザーが取得し得る点は許容済み**（UI では非表示。安全情報であり候補が見えても実害は小さい、との判断）。

## 5. 公開API（P2）

### 5.1 エンドポイント

自プロジェクトの Vercel Function として実装する（同一オリジン）。

| メソッド | パス                         | 認証 | 説明                           |
|------|----------------------------|----|------------------------------|
| GET  | <code>/api/closures</code> | 不要 | 公開ストアの最新 geojson を返す         |
| POST | <code>/api/closures</code> | 必要 | 受け取った geojson を公開ストアへ保存（全置換） |

### 5.2 保存先（公開ストア）

- **Vercel Blob を推奨**（ファイル状の geojson と相性が良い）。代替として Vercel KV。
- 保存単位は**ファイル全体の全置換**（部分更新はしない）。解除＝該当地点を含まない 新ファイルで上書きすることで地図から消える。

### 5.3 認証

- 公開トークンを環境変数（例 `CLOSURES_PUBLISH_TOKEN`）に設定する。
- POST 時にヘッダ（例 `x-publish-token`）で照合し、不一致は **401** を返す。
- トークンは運用担当者のみが知る。クライアントには埋め込まず、preview 時に入力する。

### 5.4 入力バリデーション（POST）

- FeatureCollection であること、各 Feature が「4.2 スキーマ」を満たすこと、`id` の重複が無いこと、座標が箕面エリアの妥当な範囲であることを検証し、不正は **400**。
- サーバ側で `FeatureCollection.updatedAt` を保存時刻で付与する。

### 5.5 レスポンス

- 成功：**200**（保存件数・`updatedAt` を返す）
- 認証失敗：**401** / 入力不正：**400** / その他：**500**

## 6. キャッシュ／オフライン（Service Worker）

- `/api/closures` を `SHELL_LOCAL_PATHS` に含めない。
- 取得戦略は **network-first**（オンライン時は毎回最新を取得）+ **専用キャッシュ**（例 `closures-cache`）へ保存し、**オフライン時は最終取得をフォールバック表示**。
- これにより、登山口など電波のある場所で最新を取得 → 山中ではキャッシュ参照、という既存のオフライン運用に整合する。

- 画面に\*\*「最終更新日時」\*\*を表示し、オフライン時の鮮度を判断できるようにする。

---

## 7. preview モード（運用担当者向け）

### 7.1 モードの開始・終了

- URL `?preview=closures` で開くと localStorage フラグ（例 `minoh-hiking.preview-closures = '1'`）を立て、インストール済み PWA でも持続させる。
- preview 中であることのインジケータと、解除操作（例 `?preview=off` またはボタン）を用意する。
- 一般ユーザーは通常 URL では preview 機能に到達しない。

### 7.2 preview 時のみ表示する UI

1. **ファイル取り込み** (`<input type="file" accept=".geojson,.json">`)
  - 取り込んだ geojson を解析・検証し、地図に\*\*全地点（draft 含む）\*\*を描画。
  - 作業データは端末ローカル（メモリ / IndexedDB の作業コピー）に保持。**他ユーザーには波及しない。**
2. **「公開」ボタン**
  - 公開トークンを入力（または保存済みを使用）。
  - 取り込み済み geojson を `POST /api/closures` へ送信し、公開ストアを更新。
  - 成功後、一般ユーザーは次のフレッシュ取得で **published** 地点を閲覧できる。

### 7.3 検証と公開の関係

- ファイル取り込みは**端末ローカルのみ**であり「検証専用」。公開には必ず「公開」ボタン（API 送信）が必要。
- 「テスト→公開」は、対象地点の **status** を **published** にした geojson を取り込み→検証→公開することで成立する。**status の設定・編集方法は「9. 保留事項」とする。**

---

## 8. 表示UI（一般ユーザー）

- **レイヤー表示トグル**：マップのメニューパネルに「通行止め・通行困難を表示」を追加（既定 ON）。
- **マーカー**：`config.js` の `MARKER_TYPES` に closures 用の警告マーカーを追加（**closed** と **difficult** を視覚的に区別。色・形状は設定で変更可）。
- **ポップアップ**：名称・種別（通行止め/通行困難）・理由・補足・更新日を表示。
- **件数表示**（任意）：既存の件数パネルに通行止め等の件数を追加検討。

---

## 9. 保留事項（要・運用担当者との協議後に設計）

以下は今回**保留**とし、運用手順を協議のうえ別途設計する。

- **status フラグの編集場所**：別アプリ（TrailMapper系）で設定するか、本アプリの preview 検証機能内で編集（地点ごとに公開/非公開トグル）できるようにするか。
- **取り込み時の id マージ方針**：TrailMapper 再出力のたびに過去の公開判断（status）を失わないために、公開ストアと取り込みファイルを **id** で突合（既存 id は status 継承・座標等は更新、新規は draft、欠落 id は削除候補）する方式の採否。
- 上記に伴う作業コピーの保持・中断/再開の仕様。

本書の「7. preview モード」「4.4 公開可否」は、`status` が正しく設定済みの `geojson` が取り込まれる前提で記述している。`status` の設定・編集・マージの具体は保留事項の決定後に追記する。

## 10. アプリ側の変更想定（実装時・参考）

| ファイル                                  | 変更概要                                                                                                                 |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>public/config.js</code>         | closures 取得URL ( <code>/api/closures</code> )、preview/トークン用 localStorage キー、 <code>MARKER_TYPES</code> に closures 追加 |
| <code>public/map.js</code>            | closures レイヤー（既存 <code>loadEmergencyPointsLayer</code> と同型）。一般時は <code>status==='published'</code> で filter          |
| <code>public/app.js</code>            | ? <code>preview=closures</code> 検出・フラグ保存、preview 時のみ「取り込み」「公開」ボタンと解除UI、公開 POST 処理                                    |
| <code>public/service-worker.js</code> | <code>/api/closures</code> を <code>SHELL_LOCAL_PATHS</code> に入れず network-first + 専用キャッシュ + オフラインフォールバック              |
| <code>public/index.html</code>        | 「通行止め・通行困難を表示」トグル、preview 用 UI                                                                                       |
| <code>api/closures.*</code><br>(新規)   | Vercel Function (GET/POST)、Blob/KV 連携、トークン認証、バリデーション                                                                 |
| Vercel 設定                             | 公開トークンの環境変数、Blob/KV の有効化                                                                                             |

## 11. セキュリティ・運用上の考慮

- 公開トークンはクライアントに埋め込まない。運用担当者が preview 時に入力する。
- POST は必ずトークン照合し、サーバ側でスキーマ検証する（不正データの公開を防止）。
- 公開は全置換のため、誤って空ファイルを送ると全消去になる。送信前に件数確認や 確認ダイアログを設けることを推奨。
- draft が API 応答に含まれることは許容済みだが、機密が必要になった場合は別ファイル分離（旧案C）を再検討する。

## 12. 対象外（当面）

- 別アプリ（TrailMapper系）側の改修。
- ルート単位の「迂回案内」「自動う回路計算」等のナビ連携（`relatedRoute` は将来用に予約のみ）。
- 公開履歴・変更ログ・複数運用者の同時編集制御。

## 13. git/GitHub 運用とコード・データの分離

P2 を採用した結果、「コード」と「実データ」は GitHub の扱いが正反対になる。本節はその境界と運用フローを明示する。

### 13.1 基本原則：GitHub は「コード」だけを管理する

設計書 2.2 の「データ更新は git / deploy を一切経由しない」を、配布物の観点で言い換えると「**closures の実データは GitHub に載らない**」となる。これは既存 `minoh-hiking-routes-spots.geojson` (git 管理 + アプリシェル同梱) との決定的な違いである。

| 種類            | 中身                                                                                                                                                                         | 置き場所                   | git/GitHub                      |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|---------------------------------|
| コード           | Function 実装・ <code>map.js</code> ・ <code>app.js</code> ・ <code>config.js</code> ・ <code>service-worker.js</code> ・ <code>index.html</code> ・本設計書・ <code>vercel.json</code> | リポジトリ                  | ○ git 管理 → push → Vercel 自動デプロイ |
| closures 実データ | 公開された通行止め・通行困難地点の geojson                                                                                                                                                  | 公開ストア (Vercel Blob/KV) | × git 非経由 (API で直接更新)           |
| 秘密情報          | 公開トークン ( <code>CLOSURES_PUBLISH_TOKEN</code> )                                                                                                                             | Vercel 環境変数            | × コミットしない                       |

## 13.2 2つのフローの分離

### ① 機能の開発・修正 (開発担当・git 経由)

コード変更 → commit → GitHub push → Vercel 自動デプロイ

対象: `api/closures.*` (新規)、`map.js` / `app.js` / `config.js` / `service-worker.js` / `index.html` の改修、`vercel.json`、本設計書。従来の開発フローと同一。

### ② データの公開 (運用担当・git 非経由)

preview 画面で取り込み → 検証 → 「公開」ボタン → POST `/api/closures` → Blob/KV

ここには git・GitHub・Vercel デプロイは一切登場しない。これが「運用担当の端末に git 不要・1ボタン完結」(設計書 1.2) を満たす理由である。

## 13.3 運用上の注意

- **再デプロイで実データは消えない**：実データは公開ストア (リポジトリ外) にあるため、コード修正による再デプロイの影響を受けず、公開済みデータは保持される (P2 の利点)。
- **トークン・環境変数はコミット禁止**：`CLOSURES_PUBLISH_TOKEN` は Vercel 環境変数のみで設定。`.gitignore` に `.env*` を含める。`vercel.json` には Blob/KV 利用設定は書いても**値は書かない**。
- **初回デプロイ時の空ストア対策**：誰も公開していない状態で `GET /api/closures` が呼ばれてもエラーにせず空の `FeatureCollection` を返す (Function 側で処理)。リポジトリへの初期データ配置は不要。
- **開発用サンプルの扱い**：ローカル検証用に `fixtures/closures.sample.geojson` 等を リポジトリに置くのは可。ただし「**配信されない・dev 用**」と明示し、本番取得元が API である点と区別する。
- **ブランチ運用**：実装は `feat/closures` 等のブランチを切り PR 運用とする (②のデータ公開とは無関係)。

### 13.4 データ履歴を git に求めない

git は公開データの履歴を持たない（公開履歴は設計書 12 で対象外）。監査ログ等が必要になった場合は「公開時にスナップショットを別途残す」仕組みを追加で検討する。P2 のまま GitHub に履歴を求めると、却下した P1（git push 公開）の発想に逆戻りするため、両者を混在させない。